

Development



This project required several aspects of product development outside of the realm of pure computer science, such as electrical engineering and physical construction/design. I was in charge of those parts of the process and also contributed foundational parts of the codebase that allowed our **Arduino-based synth** to convert input into output sounds.

We had ~4 months for the entire project.

For example, a specific feature that I developed was the “octave bend” mode, which allows the player to shift the octave of a key pressed by pushing the key left or right. This effectively triples the number of notes our synth-keyboard can play.

```
if (abs(yValue) > bendBuffer){
  if (yValue < 0){
    soundGen.setFrequency(keys[i].voice, noteFrequency(baseNote + i + 12));
  }else{
    soundGen.setFrequency(keys[i].voice, noteFrequency(baseNote + i - 12));
  }
}
```

This code snippet is the essential part of this feature. For each key, if the horizontal input is greater than a certain buffer the key's frequency will be adjusted by twelve notes (an octave) up or down based on the direction that the key is pushed.

Each physical key is actually a joystick with a 3D-Printed key attached to it. This allows us to easily send both vertical and horizontal input to the arduino-brain of our project that takes the input and generates the sound.



I modeled the final physical body of our instrument in Fusion and took it to our Engineering facilities at RHS to laser cut the sides of our box, which I then put together. We stuffed our wire jungle into the case, making our project presentable.

Reflection and Takeaway

A major part of this project was about learning how to make software integrate with custom hardware. This not only required us to familiarize ourselves with the electrical side of computing devices and product design, but also gave us a new appreciation of how the devices that we use interface with the user. **I discovered that I'm capable of combining different trades and skills to create something beyond the scope of a singular field.**

Specific things I learned from this project were: laser cutting (have a look at the instrument's exterior), basic electronics and breadboarding (take off the lid to take a peek), and a deeper understanding of music and sound generation; hit a note and hear it ring!



Kavan Abayanayaka
Roseville High, Computer Science,
Technology Innovation



What is it?

Do you play the **piano**?

Have you ever played the **piano**?

Well, if you have, you may have experienced the urge to simply **push the key left and right to change the pitch of a note**. Imagine if each key was a whammy bar, allowing for satisfying expression with each note.



Dream no more! The *Wamboard* is exactly that. Each key has horizontal input, allowing you to alter the quality of each note played with a simple wiggle.

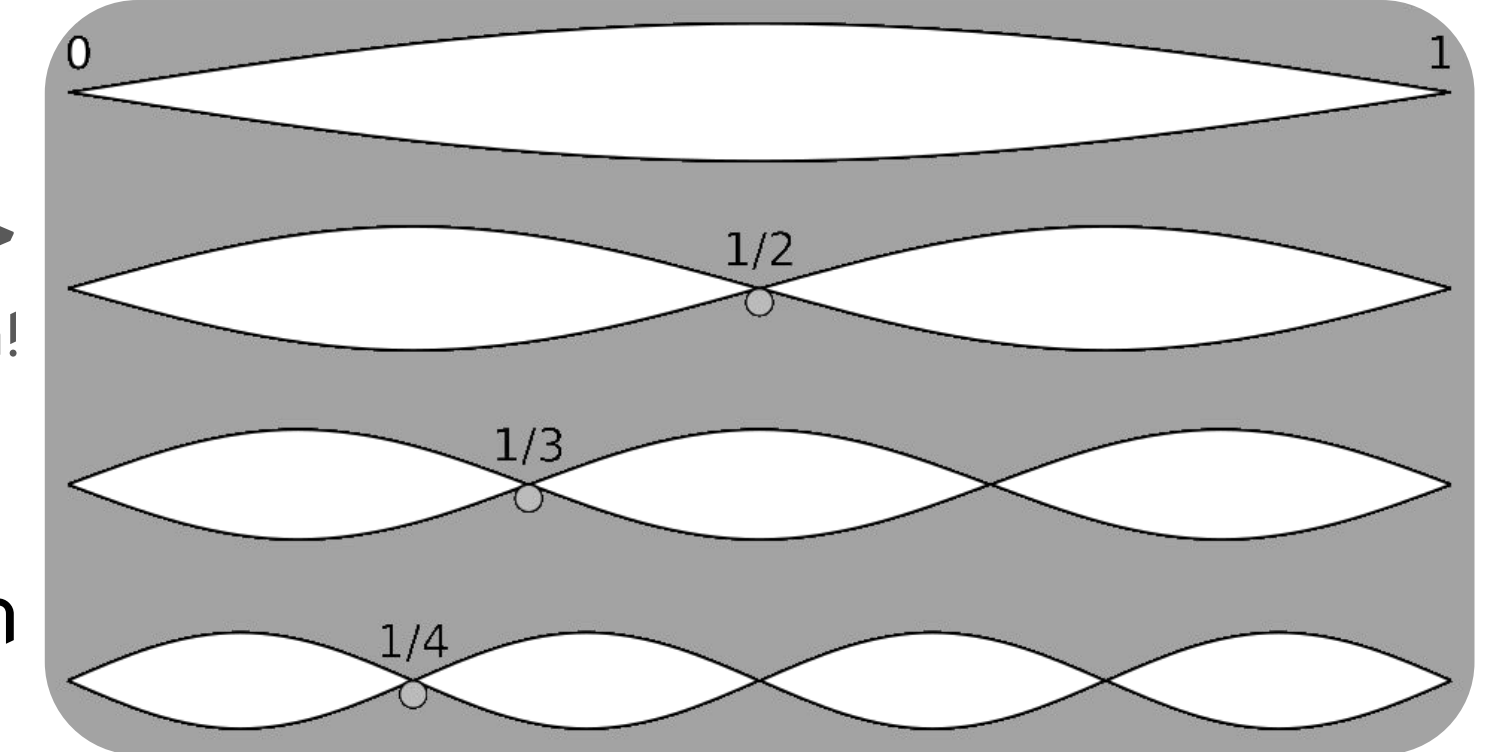


A Problem... And Solution!

We were pretty happy when we managed to make simple notes generated by our Arduino, but the notes sounded boring, and not particularly pleasant or interesting. This wasn't (entirely) reflexive of our ability to play the instrument, but largely due to the primitive and boring waveforms being played.

Overtones! →
Each is a higher pitch!

Any sound can be broken down into a set of combined sine waves of different frequencies. Musical instruments, unlike our instrument at the time, don't only produce one sine wave, but a set of sine waves of different frequencies. Every instrument has a different set of these that make their unique sound. These are called *overtones*.

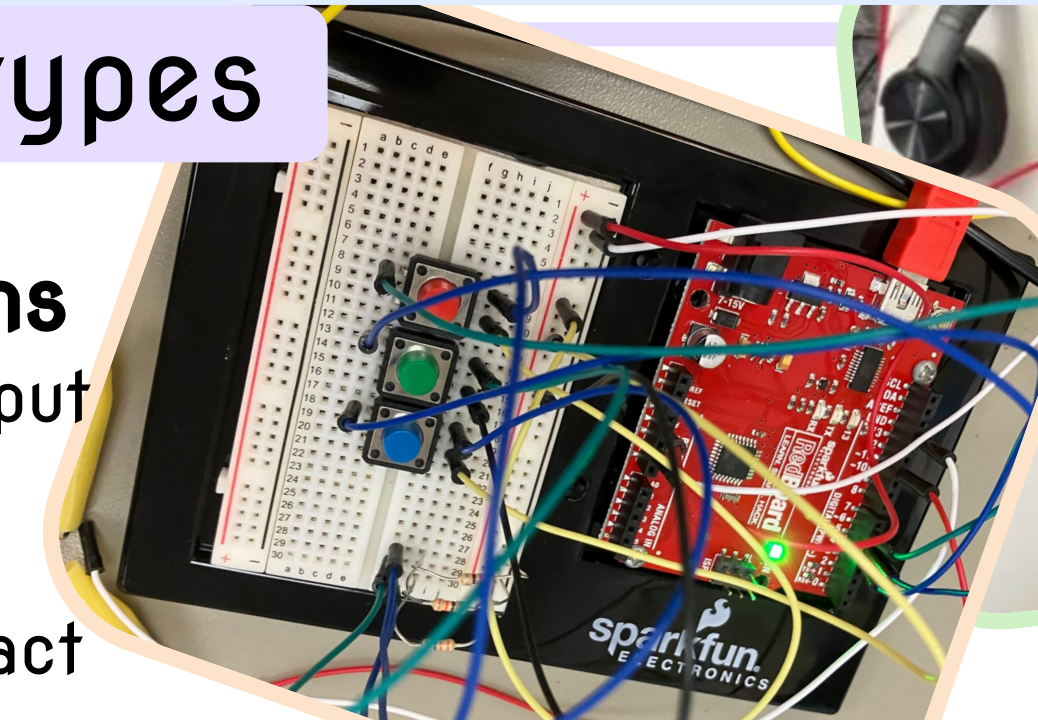


To allow our instrument to play more complex and somewhat “realistic” instrument sounds, I created a Python script that takes in volume parameters for each overtone that should be added to the sound and outputs a wavetable (a set of numbers representing a sound wave that our synth can adjust and output as audio signals) generated from those overtones.

Prototypes

3 Buttons

Audio output involves handheld wire contact



Key Test

6 keys with horizontal & vertical input

